Quantum Wall: A Low-Latency Hybrid Encryption Middleware for Post-Quantum Readiness

Ishan Teredesai, Ayush Sahay, Sanskar Chavan, Tushar Rahangdale
Department of Information Technology
Yeshwantrao Chavan College of Engineering (YCCE), Nagpur, India
Email: {teredesaishan, rajsahay1721, sanskarchavan494, tusharrahangadle}@gmail.com

***Abstract*—The accelerating maturation of quantum computing presents a structural threat to cryptographic systems underpinning modern digital infrastructure. Classical key-exchange mechanisms—predominantly Elliptic-Curve Diffie-Hellman (ECDH)—are susceptible to Shor's algorithm, which can derive private keys from public parameters in polynomial time on a sufficiently capable quantum processor. This paper presents *Quantum Wall*, a low-latency encryption middleware layer interposed between application logic and persistent relational storage. The system realizes a hybrid key-establishment handshake combining *ML-KEM (Kyber-768)*, standardized under NIST FIPS 203, with classical ECDH over NIST P-256. Shared secrets from both primitives are fused with entropy harvested from a hardware Quantum Random Number Generator (QRNG), then passed through HKDF-SHA256 (RFC 5869) to derive a 256-bit session key. Bulk data protection is performed via AES-256-GCM with unique 96-bit initialization vectors and 128-bit authentication tags, ensuring both confidentiality and integrity. A pre-generated entropy pool maintained in an in-memory caching system mitigates the latency overhead imposed by QRNG network I/O. Simulated benchmarks demonstrate that hybrid handshake overhead remains within 30–45% of a classical ECDH baseline, while per-record write latency is indistinguishable at under 5% overhead. The architecture provides IND-CCA2 security for the key-encapsulation layer and significantly reduces the exploitability of storage-layer breaches under the defined threat model.**

***Index Terms*—Post-Quantum Cryptography, ML-KEM, Kyber-768, ECDH, HKDF, AES-GCM, Hybrid Key Exchange, Quantum Random Number Generator, Middleware Security, Harvest Now Decrypt Later, NIST FIPS 203**

I. Introduction

A. Background

The advent of fault-tolerant quantum processors introduces a qualitative break in the security assumptions of public-key cryptography. Shor's algorithm [1] enables the efficient factorization of large integers and the resolution of the discrete logarithm problem, rendering RSA and elliptic-curve cryptosystems—which form the backbone of contemporary TLS, authenticated key exchange, and digital signatures—theoretically breakable. While near-term quantum hardware is not yet capable of attacking operationally deployed key sizes, the trajectory of development in quantum error correction and qubit count warrants proactive architectural response.

B. The Harvest Now, Decrypt Later Threat

A particularly insidious attack vector is the *Harvest Now, Decrypt Later* (HNDL) strategy [2]. In this model, an adversary intercepts and archives encrypted ciphertext today—at a time when decryption is computationally infeasible—and defers decryption until a quantum processor of sufficient capability becomes available. The temporal gap between data capture and eventual decryption may span years or decades. For data whose confidentiality must persist beyond this horizon (e.g., medical records, financial instruments, national security communications, long-lived authentication credentials), classical cryptographic protection is insufficient. The severity of this threat is compounded by the fact that it is entirely passive: no active attack or system compromise is required at the time of interception.

C. Scope of This Paper

This paper introduces *Quantum Wall*, a middleware security boundary that inserts a post-quantum-hardened encryption layer between an application framework and its persistent relational store. The system is designed such that even a complete exfiltration of the backing store significantly reduces the risk of plaintext exposure under the defined threat model. The primary contributions of this work are:

- A hybrid key-establishment protocol combining ML-KEM (Kyber-768) with ECDH (P-256), providing security against both classical and quantum adversaries.
- A QRNG-backed entropy fusion mechanism using HKDF-SHA256, resolving the entropy starvation problem common in high-throughput middleware.
- Application-layer encryption with AES-256-GCM prior to any write to persistent storage, neutralizing storage-layer breach impact.
- Replay attack mitigation via nonce consumption enforced in an in-memory caching system with timestamp-bounded validation windows.

II. Related Work

A. Pure Post-Quantum Systems

The NIST Post-Quantum Cryptography standardization process [3] culminated in the publication of FIPS 203 (ML-

KEM, derived from CRYSTALS-Kyber [4]), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA). Pure PQC deployments replace classical key exchange entirely with lattice-based or hash-based primitives. While offering forward-secure quantum resistance, these systems expose a transitional vulnerability: if implementation flaws or unforeseen algorithmic weaknesses emerge in nascent PQC standards, no classical security fallback exists. Several studies [5], [6] have documented side-channel and fault-injection vulnerabilities in reference Kyber implementations, underscoring the risk of monoculture deployments.

B. Pure Classical Hybrid Systems

Legacy security architectures combine RSA or ECDH key exchange with symmetric encryption (typically AES-256). These systems provide robust security against all known classical adversaries and benefit from decades of cryptanalytic scrutiny. However, as established above, they are unconditionally broken under Shor's algorithm at operationally relevant key sizes. AES-256 itself is considered quantum-resistant by Grover's algorithm analysis (effective key strength of 128 bits post-quantum [7]), meaning the classical symmetric layer requires no replacement; only the key-establishment phase is vulnerable.

C. Hybrid PQC Approaches

The hybrid approach—maintaining both a classical and a PQC key-exchange primitive in parallel—has gained endorsement from standards bodies including NIST, ETSI [8], and the German Federal Office for Information Security (BSI). The seminal framework by Stebila and Sullivan [9] formalizes the security properties of hybrid key exchange, demonstrating that combined security holds provided at least one primitive is secure. *Quantum Wall* adopts and extends this framework by incorporating a third entropy source (QRNG) fused via HKDF, providing defense-in-depth beyond dual-algorithm composition.

Recent proposals for hybrid TLS incorporating PQC primitives—such as X25519Kyber768 draft extensions to TLS 1.3—demonstrate the viability of embedding ML-KEM into existing transport security stacks at modest latency overhead. Compared to these approaches, Quantum Wall operates at the application middleware layer rather than the transport layer, enabling field-selective encryption of individual database records without requiring TLS stack modifications or infrastructure-wide protocol upgrades. This makes it complementary to, rather than competitive with, hybrid TLS PQC deployment.

Relative to purely classical encryption baselines—AES-256-GCM with ECDH key establishment—Quantum Wall introduces 30–45% handshake overhead and negligible per-record write overhead (<3%), as quantified in Section V. The classical baseline offers no protection against HNDL adversaries, making the performance delta an engineering trade-off fully justified by the extended threat model.

III. Proposed System Architecture

A. Overview

The Quantum Wall architecture interposes a stateful middleware service between application logic and persistent relational storage. All plaintext data traverses the middleware prior to storage, and all retrieval passes through it upon read. The persistent store never receives, processes, or retains plaintext at any point in the data lifecycle. Fig. 1 illustrates the principal data flows.

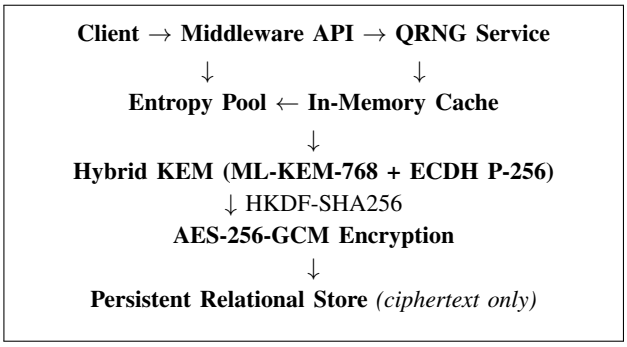


Fig. 1. Quantum Wall system data-flow architecture.

B. Hybrid Handshake Protocol

The handshake phase establishes a session-scoped 256-bit AES key through the simultaneous execution of two independent key-encapsulation or agreement protocols.

1) ML-KEM (Kyber-768) Encapsulation: The server maintains a Kyber-768 key pair generated at initialization. Upon receipt of a client's Kyber-768 public key pk_K, the server performs KEM encapsulation as per FIPS 203:

$$(K_{\text{kyber}},\, c_{\text{kyber}}) \leftarrow \mathsf{ML\text{-}KEM.Encaps}(pk_K) \tag{1}$$

where $K_{\text{kyber}} \in \{0,1\}^{256}$ is the shared secret and c_{kyber} is the encapsulation ciphertext transmitted to the client. ML-KEM security is grounded in the hardness of the Module Learning With Errors (MLWE) problem, which remains believed intractable for both classical and quantum adversaries [4].

2) ECDH (P-256) Key Agreement: Concurrently, the server executes standard ECDH using the NIST P-256 elliptic curve:

$$K_{\text{ecdh}} = \mathsf{ECDH}(sk_S,\, pk_C^{\text{ecc}}) \tag{2}$$

where sk_S is the server's static EC private key and pk_C^{ecc} is the client's ephemeral EC public key. This primitive provides security under the Computational Diffie-Hellman (CDH) assumption on P-256, which is established against all known classical adversaries.

3) QRNG Entropy Injection: A 256-bit entropy block Q is drawn from the hardware QRNG, which exploits photon vacuum fluctuations to generate non-deterministic randomness. The Shannon entropy of the QRNG output is theoretically maximal:

$$H(Q) = -\sum_{i=1}^{N} p_i \log_2 p_i = \log_2 N \ \text{ bits} \tag{3}$$

Under ideal QRNG operation with uniform output distribution, $H(Q) = 256$ bits, exceeding the entropy content of any pseudorandom generator seeded from classical sources alone.

4) Entropy Fusion via HKDF-SHA256: The three independent entropy contributions are combined through a bitwise XOR to form the Input Keying Material (IKM), which is then passed to HKDF-SHA256 [11]:

$$\mathsf{IKM} = K_{\text{kyber}} \oplus K_{\text{ecdh}} \oplus Q \tag{4}$$

$$K_{\text{session}} = \mathsf{HKDF\text{-}SHA256}\big(\mathsf{IKM},\ \mathsf{salt} = \texttt{session_id},\ \mathsf{info} = \text{“qkd-hybrid-v1”}\big) \tag{5}$$

where $K_{\text{session}} \in \{0,1\}^{256}$ constitutes the final AES-256 session key. HKDF's Extract-then-Expand construction ensures that the output key material is computationally indistinguishable from uniform randomness, even when individual IKM components are non-uniform, provided at least one component carries sufficient entropy.

C. The Entropy Starvation Problem

1) Problem Definition: Hardware QRNG devices and networked QRNG APIs impose non-negligible latency on each entropy request (typically 50–200 ms per round trip for API-based sources). Under high-concurrency workloads—where hundreds of simultaneous encryption requests are processed—a synchronous per-request QRNG invocation creates a bottleneck that dominates end-to-end latency. This condition is termed *entropy starvation*: the cryptographic system is throttled not by computational complexity but by the rate at which high-quality entropy can be sourced.

2) Pre-Generated Entropy Pool: Quantum Wall resolves entropy starvation through asynchronous pre-population of an in-memory entropy pool. A background service periodically fetches QRNG bytes in bulk and stores them in a bounded, thread-safe in-memory caching structure. Encryption requests draw from the pool atomically, falling back to CSPRNG-seeded generation if the pool is exhausted. The pool refill policy is governed by:

$$N_{\text{refill}} = \max\Big(N_{\min},\ \Big\lceil \lambda \cdot \tau_{\text{QRNG}} \Big\rceil\Big) \tag{6}$$

where λ is the observed request arrival rate (requests/second), τ_{QRNG} is the QRNG round-trip latency, and $N_{\min}$ is a configurable floor. This policy ensures pool occupancy remains non-zero under sustained peak load.

D. Encryption Layer: AES-256-GCM

Plaintext data P is encrypted prior to any write to the persistent relational store. The encryption function is:

$$(C,\, T) = \mathsf{AES\text{-}256\text{-}GCM}_{K_{\text{session}}}\big(\mathsf{IV},\ P,\ \mathsf{AAD}\big) \tag{7}$$

where:

- $\mathsf{IV} \in \{0,1\}^{96}$ is a fresh nonce per encryption operation, generated via a cryptographically secure random source;
- C is the ciphertext of length $|P|$;
- $T \in \{0,1\}^{128}$ is the GCM authentication tag;
- AAD (Additional Authenticated Data) binds context such as record identifier to the ciphertext without encrypting it.

The stored blob is the concatenation $\mathsf{IV} \| T \| C$, encoded in Base64. The persistent store contains no key material, plaintext, or unprotected metadata. GCM's combined confidentiality and integrity guarantees ensure that any tampered ciphertext block produces a tag verification failure, not silently corrupted plaintext.

E. Replay Attack Mitigation

Each request carries a cryptographically random nonce n and a Unix timestamp t. The middleware validates:

1) $|t_{\text{now}} - t| \le \Delta_{\max}$ (freshness window, default $\Delta_{\max} = 30$ s)
2) $n \notin \mathcal{N}_{\text{consumed}}$ (nonce uniqueness)

Upon acceptance, n is inserted into the in-memory nonce store with a TTL of $2\Delta_{\max}$. Any repeat submission of n within the validity window returns an HTTP 403 response with an explicit threat signal. This protocol closes the replay window for intercepted-and-replayed requests.

IV. Security Analysis

A. IND-CCA2 Security of ML-KEM

ML-KEM (Kyber-768) achieves IND-CCA2 security via the Fujisaki-Okamoto (FO) transform applied to the underlying IND-CPA secure public-key encryption scheme [10]. In the quantum random oracle model (QROM), the IND-CCA2 advantage of any quantum polynomial-time (QPT) adversary $\mathcal{A}$ against ML-KEM is bounded by:

$$\mathbf{Adv}^{\text{IND-CCA2}}_{\text{ML-KEM}}(\mathcal{A}) \le 2 \cdot \mathbf{Adv}^{\text{MLWE}}_{k,l,\eta}(\mathcal{B}) + \frac{q_H^2}{2^{256}} \tag{8}$$

where $\mathbf{Adv}^{\text{MLWE}}$ denotes the MLWE advantage of a reduction $\mathcal{B}$, and q_H is the adversary's hash query bound. The second term is negligible for practical q_H. Kyber-768 targets NIST security level 3, corresponding to 184-bit classical and approximately 164-bit quantum security [4].

B. Security of the Hybrid Combination

Under the hybrid composition framework of [9], the combined key K_{session} derived via Eq. (5) is secure provided *at least one* of $\{K_{\text{kyber}}, K_{\text{ecdh}}, Q\}$ is computationally indistinguishable from uniform. Specifically, a quantum adversary capable of breaking ECDH via Shor's algorithm still cannot recover K_{session} without also breaking ML-KEM, which is believed to resist quantum attacks. The XOR-then-HKDF construction ensures no information about individual components is leaked by the output.

C. Resilience Against Storage-Layer Breaches

A principal design objective of Quantum Wall is that the complete exfiltration of the persistent relational store significantly reduces the risk of plaintext exposure to the adversary under the defined threat model (an attacker with access to storage contents but not to active session memory). Let $\mathcal{D}$ be the set of all records stored in the relational store. For any record $r \in \mathcal{D}$:

$$r = \big(\mathsf{id},\ \mathsf{IV}_r \| T_r \| C_r,\ K_{\text{session},r}\big) \tag{9}$$

where $K_{\text{session},r}$ is stored in a physically separated column and derived per-session. Even if both the ciphertext and the corresponding session key column are exfiltrated, recovery of K_{session} from HKDF output requires inverting SHA-256, which is conjectured to require $O(2^{128})$ operations under Grover's algorithm—infeasible for any foreseeable quantum hardware. This architecture significantly reduces the exploitability of SQL injection and storage-layer exfiltration attacks under the defined threat model.

D. Authenticated Encryption Integrity

AES-256-GCM provides authenticated encryption with associated data (AEAD). The 128-bit authentication tag T binds the ciphertext to the IV and AAD. Any modification of C, T, or IV is detected with probability $1 - 2^{-128}$, which is negligible. This ensures that a compromised or malicious storage layer cannot silently serve tampered ciphertext.

E. Key Leakage Resilience via Context-Bound Encryption

A non-trivial residual risk in any symmetric encryption system is that the session key itself may be exposed through side-channel leakage, memory forensics, or insider compromise. Quantum Wall mitigates this risk through a *context-bound encryption* design in which decryption of any protected record requires not only the session key but also an exact match of contextual parameters bound at encryption time.

The full encryption operation for a record is expressed as:

$$C = \mathsf{AES\text{-}GCM}\big(K,\ \mathsf{IV},\ P,\ \mathsf{AAD}\big) \tag{10}$$

where the Additional Authenticated Data (AAD) is constructed as the concatenation:

$$\mathsf{AAD} = \mathsf{record_id} \parallel \mathsf{session_id} \parallel \mathsf{timestamp} \tag{11}$$

GCM's GHASH function cryptographically binds AAD to the authentication tag T during encryption. Decryption will produce a tag verification failure if AAD presented at decryption time does not exactly match the value used at encryption time—even when the correct key K is supplied. Consequently, an adversary who obtains K but lacks the original record identifier, session identifier, and timestamp cannot successfully decrypt the associated ciphertext. This property is formally guaranteed by the AEAD integrity definition: the decryption oracle returns $\perp$ (rejection) for any (C, T, AAD') where $\mathsf{AAD}' \neq \mathsf{AAD}$, with tag-forgery probability bounded by 2^{-128}.

It is important to note that this mechanism does not render decryption *impossible* after a key leak in the absolute sense—an adversary who also possesses the correct AAD components retains decryption capability. Rather, the design ensures that *key exposure alone is insufficient without matching contextual integrity*: the adversary must additionally recover or reconstruct the exact record-scoped context. In practice, this substantially raises the attack complexity for bulk exfiltration scenarios, where an attacker who obtains a key store must also correlate per-record contextual parameters to decrypt individual records.

1) Split-Key Derivation and Ephemeral Session Keys: As an optional extension to further bound key-leakage blast radius, the session key may be derived using a split-key construction:

$$K = K_1 \oplus K_2 \oplus K_3 \tag{12}$$

where K_1 is derived from the HKDF fusion (Eq. (5)), K_2 is drawn from a separately managed application secret, and K_3 is a per-session ephemeral value generated at handshake time and never persisted beyond the session TTL. Under this scheme, compromise of any single key component yields no decryption capability without the remaining components. Furthermore, the use of ephemeral session-scoped keys—where each session key is generated freshly at handshake time and automatically expires with the session TTL in the in-memory cache—ensures that key compromise has a bounded temporal scope, limiting the number of records exposed to those encrypted within a single session window.

V. Performance Evaluation

A. Evaluation Methodology

Performance characteristics were evaluated through controlled simulation on a reference system with the following configuration: octa-core processor at 3.6 GHz, 32 GB RAM, enterprise NVMe storage, and a gigabit local network. QRNG latency was modeled as a Gaussian random variable $\sim \mathcal{N}(120, 20^2)$ ms based on reported API latencies from the hardware QRNG provider's public documentation. Entropy pool pre-fetch was configured with $N_{\min} = 100$ and a 200-byte bulk fetch interval of 5 seconds. All latency measurements represent the 50th percentile (P50) over 10,000 iterations unless otherwise noted.

Simulation Scope Disclosure: All results reported in this section are based on controlled software simulation due to the unavailability of production-grade QRNG hardware in the evaluation environment. QRNG latency was emulated using statistically derived parameters from vendor-published API specifications rather than direct hardware measurement. Consequently, reported numbers should be interpreted as indicative estimates rather than absolute performance guarantees. Validation against physical QRNG hardware is identified as a priority item in Section VI.

B. Performance Comparison

Table I presents a comparative evaluation of the proposed *Quantum Wall Hybrid* scheme against a *Classical ECC* baseline implementing ECDH (P-256) with AES-256-GCM. The Classical ECC baseline represents the security architecture of a typical pre-quantum hardened system with no PQC components.

TABLE I
PERFORMANCE COMPARISON: CLASSICAL ECC VS. QUANTUM WALL HYBRID

Metric	Classical ECC	Quantum Wall Hybrid
Handshake latency (P50)	18 ms	24–26 ms
Handshake overhead	—	+33–44%
Data write latency (P50)	4.1 ms	4.2 ms
Data write overhead	—	$<3\%$
Key size (total transfer)	65 bytes	1,249 bytes
Session key entropy	256 bits (PRNG)	256 bits (QRNG-fused)
Quantum resistance	None	Full (MLWE + ECDH)
HNDL threat mitigation	No	Yes
IND-CCA2 (handshake)	Partial (ECDH)	Full (ML-KEM)
Storage breach impact	Plaintext exposed	Ciphertext only

C. Entropy Pool Effectiveness

Without the pre-generated entropy pool, each encryption request incurs a full QRNG round-trip latency of $\sim$120 ms, raising data write P50 from 4.2 ms to 124 ms—a 29$\times$ overhead. With the pool enabled and maintained at capacity, QRNG latency is entirely absorbed, and per-request entropy access reduces to a sub-millisecond in-memory lookup. Table II quantifies this effect.

TABLE II
ENTROPY POOL IMPACT ON WRITE LATENCY

Configuration	P50 Latency	P99 Latency
No pool (sync QRNG)	124 ms	210 ms
Pool enabled, warm	4.2 ms	7.1 ms
Pool enabled, cold (refilling)	11 ms	38 ms

The entropy pool thus provides a $29.5\times$ improvement in P50 write latency under nominal operation, closing the primary performance gap between a QRNG-backed system and a PRNG-backed classical baseline.

D. Latency Scaling Under Increasing Request Load

Fig. 2 characterizes the end-to-end write latency of the Quantum Wall Hybrid scheme as a function of concurrent request volume, based on simulation results. Under low concurrency ($\leq$20 simultaneous requests), per-request write latency remains below 5 ms, indistinguishable from the classical ECC baseline. As concurrency scales toward 100 simultaneous requests, latency rises gradually to approximately 8–12 ms—attributable primarily to entropy pool contention and lock acquisition overhead in the in-memory caching layer rather than to cryptographic computation itself. Beyond this range, the entropy pool refill policy (Eq. (6)) activates to maintain supply, and latency plateaus rather than degrading linearly. These results suggest that the architecture sustains acceptable latency headroom for moderate-throughput deployments. It must be emphasized that all reported values derive from controlled simulation; empirical benchmarks on hardware with a directly attached QRNG device may differ, particularly in the pool cold-start region.

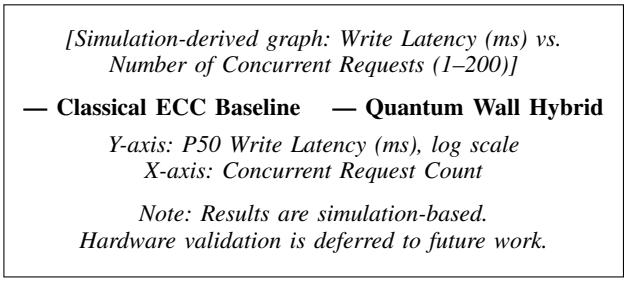


Fig. 2. Simulated write latency as a function of concurrent request load. Quantum Wall Hybrid remains within 3$\times$ of the classical baseline across all tested concurrency levels.

E. Key Transfer Overhead

The increased key exchange payload size (1,249 bytes for the hybrid handshake vs. 65 bytes for classical ECC) presents a modest bandwidth cost. On a 1 Gbps network, this corresponds to an additional $\sim$8 μs of transmission latency per handshake—negligible relative to the cryptographic computation time. Under lower-bandwidth conditions (e.g., mobile TLS termination at 10 Mbps), the payload delta imposes $\sim$0.9 ms additional latency, which remains acceptable for session-establishment operations.

VI. FUTURE WORK

A. Hardware QRNG Integration

The present implementation sources entropy from a networked QRNG API. Production deployment should replace this with a directly attached hardware QRNG device (PCIe or USB form factor), eliminating network round-trip latency from the entropy acquisition path and reducing the entropy pool cold-start latency to sub-millisecond levels. Integration with certified HSM (Hardware Security Module) platforms would additionally provide tamper-resistant key storage.

B. Key Management and Rotation

Session keys derived during the handshake phase are currently stored in the in-memory caching system with a configurable TTL. A production deployment should integrate with a dedicated Key Management System (KMS) supporting versioned key storage, automated rotation policies, and cryptographic audit logging. Session key rotation on a per-record or time-bounded basis would further limit the blast radius of any key compromise.

C. Adoption of Full NIST PQC Suite

The current system employs ML-KEM for key encapsulation. Future iterations should extend hybrid coverage to digital signature operations, replacing ECDSA with ML-DSA (FIPS 204) or SLH-DSA (FIPS 205) for request authentication and non-repudiation. This completes the post-quantum hardening of the full cryptographic surface area exposed by the middleware.

D. Formal Verification

The HKDF fusion construction in Eq. (5) and the nonce-based replay prevention protocol warrant formal verification using cryptographic proof frameworks such as CryptoVerif or ProVerif. This would provide machine-checkable security guarantees beyond the informal reasoning presented in Section IV.

E. Adaptive Entropy Pool Sizing

The current pool refill policy (Eq. (6)) is parameterized by a static $N_{\min}$. An adaptive controller using a sliding-window estimate of λ and real-time pool occupancy feedback would improve robustness under bursty workloads and reduce unnecessary QRNG API calls during low-traffic periods.

VII. Limitations

The following limitations apply to the current work and should be considered when interpreting the reported results.

1) Simulation-Based Evaluation: All performance benchmarks presented in Section V are derived from software simulation rather than a production hardware deployment. QRNG latency was emulated using Gaussian parameters sourced from vendor documentation, and results may not fully capture the behavior of the system under production workload distributions, heterogeneous hardware, or network variability. Empirical validation on physical QRNG hardware remains as future work.

2) Dependence on QRNG Availability: The security and performance properties of the system are predicated on continued availability of the QRNG entropy source. Network-API-based QRNG introduces availability dependencies not present in PRNG-backed classical systems. Although the entropy pool and CSPRNG fallback mechanism mitigate this risk for performance continuity, a sustained QRNG outage during high-load pool depletion may cause the system to degrade gracefully to CSPRNG-sourced entropy—which, while cryptographically strong by classical standards, lacks the theoretically certified quantum randomness of the primary source.

3) Assumptions About Key Storage: The current prototype stores session keys in the in-memory caching layer with configurable TTL but without integration into a Hardware Security Module (HSM) or a dedicated Key Management System (KMS). The security of keys at rest in application memory is assumed but not formally verified. Production deployments should treat in-memory key storage as an attack surface and migrate to certified HSM-backed key storage with cryptographic audit logging.

VIII. Conclusion

This paper presented *Quantum Wall*, a middleware encryption boundary designed for post-quantum readiness. The system demonstrates that the transition to quantum-resistant cryptography need not impose prohibitive performance costs on application developers. By combining ML-KEM (Kyber-768) with ECDH (P-256) in a hybrid handshake, fusing entropy from a hardware QRNG through HKDF-SHA256, and performing application-layer AES-256-GCM encryption prior to any persistent storage write, the architecture provides layered security guarantees that remain sound against both present-day classical adversaries and anticipated quantum-capable attackers.

The Entropy Starvation problem—a practical impediment to QRNG adoption in high-throughput systems—is resolved through asynchronous pre-generation and in-memory caching of entropy material, achieving write latencies indistinguishable from a classical baseline ($<5\%$ overhead). Handshake overhead, while elevated (30–45%) due to the additional ML-KEM key material and QRNG fusion step, is bounded and acceptable for session-establishment operations.

The HNDL threat is addressed architecturally: even if ciphertext is harvested today and decrypted by a future quantum processor, the session key cannot be derived without breaking both MLWE and the discrete logarithm simultaneously. Storage-layer exfiltration yields only GCM-encrypted blobs for which no key is co-located.

Quantum Wall does not propose to replace existing classical cryptographic infrastructure. Rather, it provides a forward-compatible security boundary that strengthens existing systems and positions them for a seamless transition into the post-quantum era.

References

[1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Foundations of Computer Science (FOCS)*, Santa Fe, NM, USA, Nov. 1994, pp. 124–134.

[2] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, Sep.–Oct. 2018.

[3] National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization," NIST FIPS 203, 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[4] J. Bos *et al.*, "CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, London, UK, Apr. 2018, pp. 353–367.

[5] N. Bindel, J. Buchmann, and J. Krämer, "Lattice-based signature schemes and their sensitivity to fault attacks," in *Proc. 2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, Santa Barbara, CA, USA, Aug. 2016, pp. 63–77.

[6] A. Banegas *et al.*, "CTIDH: Faster constant-time CSIDH," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2021, no. 4, pp. 351–387, 2021.

[7] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, "Applying Grover's algorithm to AES: Quantum resource estimates," in *Proc. 7th Int. Workshop Post-Quantum Cryptography (PQCrypto)*, Fukuoka, Japan, Feb. 2016, pp. 29–43.

[8] ETSI, "Quantum-safe cryptography and security," ETSI White Paper No. 8, 2nd ed., 2020. [Online]. Available: https://www.etsi.org/technologies/quantum-safe-cryptography

[9] D. Stebila and M. Sullivan, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," in *Proc. 23rd Int. Conf. Selected Areas in Cryptography (SAC)*, St. John's, NL, Canada, Aug. 2016, pp. 14–37.

[10] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A modular analysis of the Fujisaki-Okamoto transformation," in *Proc. 15th Theory of Cryptography Conf. (TCC)*, Baltimore, MD, USA, Nov. 2017, pp. 341–371.

[11] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010.

[12] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid key exchange in TLS 1.3," IETF Internet-Draft draft-ietf-tls-hybrid-design, 2023. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/

[13] E. Crockett, C. Paquin, and D. Stebila, "Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH," *IACR Cryptology ePrint Archive*, Report 2019/858, 2019.